GRAFIQ: Do VLMs Understand Image Formation?

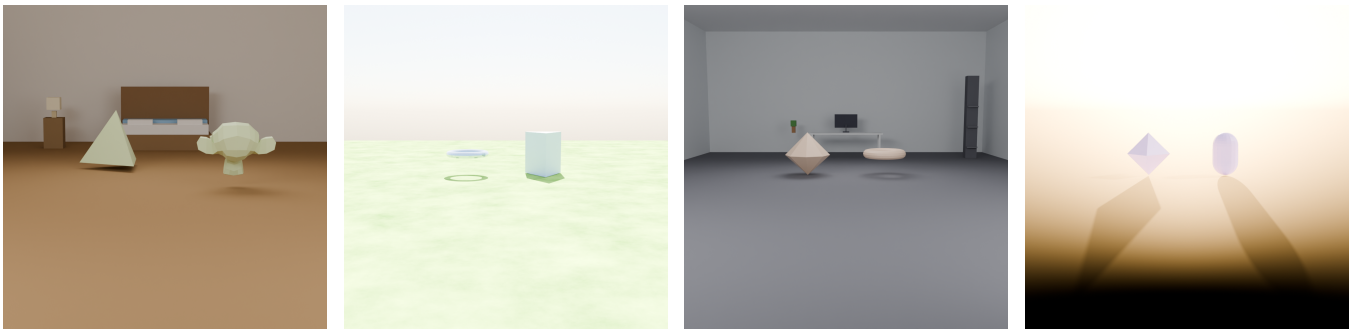
Final Project Report

Steve Chen
UC Berkeley

Patrick Tsung-Han Wu
UC Berkeley

Fei Gao
UC Berkeley

Alex Wang
UC Berkeley



(a) Geometry: depth. **(b) Materials: roughness.** **(c) Geometry: normal.** **(d) Lighting: light direction.**

Figure 1: One pixel–verified GRAFIQ item per task. Each panel shows the rendered image and the natural–language question, the correct viewer–relative `Left`/`Right` answer, and the pass–level evidence (mean camera–space depth, 95th–percentile glossy luminance, mean camera–facing–ness, and mean diffuse luminance, respectively). The four panels span the implemented axes: depth and normal orientation (geometry), roughness (materials), and light direction (lighting).

Abstract

Vision–language models (VLMs) are increasingly used as judges for image generation and visual reasoning, yet it remains unclear whether they actually understand *image formation*: the geometry, materials, lighting, and viewpoint that determine appearance. We introduce GRAFIQ (*Graphic Intelligence Quotient*), a synthetic benchmark that isolates these factors through deterministic Blender renders and viewer–relative forced–choice questions. Each item's ground truth is verified directly against render passes (depth, glossy–direct, surface normal, or diffuse–direct), so the correct answer is supported by the image evidence rather than only by the scene graph. We evaluate **seven** VLMs on a **480-item** split (four tasks × 120 items): two open local models (Qwen2.5–VL–7B–Instruct, Qwen3–VL–8B–Thinking) and five API models (GLM–4.5V, GPT–4o–mini, GPT–5–mini, Gemini 3.1 Pro (Thinking), Claude Opus 4.5), with greedy decoding and bootstrap confidence intervals. Gemini 3.1 Pro (Thinking) achieves the highest overall accuracy (74.8%), followed by GPT–5–mini (69.6%) and GLM–4.5V (69.2%); roughness and depth are substantially easier than normal orientation and light direction, and several models exhibit systematic Left/Right bias. We further benchmark **inverse generation** (scaffolded Blender Python from reference renders) on a **120-item** coding subset; scoring measures headless execution yield rather than pixel match to the reference. Gemini 3.1 Pro (Thinking) achieves the highest inverse yield (100% after minor API correction), followed by Claude Opus 4.5 (80.8%), while Qwen 3 VL and GPT minis lag behind. Together, these results separate recognition of controlled graphics cues from reliable inverse code generation: models that rank depth or gloss well can still fail often at emitting runnable scene scripts.

Code, data, and project page.
Code: https://github.com/berkeley-grafiq/grafiq_datagen
Dataset: https://huggingface.co/datasets/tsunghanwu/berkeley-grafiq-dataset
Project page: https://berkeley-grafiq.github.io/

1 Introduction

Vision–language models (VLMs) now achieve strong performance on a broad range of recognition and visual question–answering

benchmarks [1, 9, 14, 19], and are increasingly deployed as *judges* for image–generation systems, where they are asked to verify outputs, compare models, and critique rendered results. Whether these models understand the underlying process of image formation, however, remains an open question. A model that reasons primarily from image semantics can describe a scene correctly while being oblivious to whether its shading is consistent with the depicted light, whether a reported depth ordering matches the actual geometry, or whether an observed highlight is explained by the stated material. In a judging role, such failures silently propagate into the downstream generator.

Prior perceptual–probing benchmarks have exposed related gaps. BLINK [6] shows that strong VLMs fail on basic perceptual primitives such as depth ordering and reflectance estimation; PhysBench [4] documents similar failures on physical–world reasoning. These results motivate a more targeted question: can VLMs reason about the specific graphics factors that produce a rendered image? Answering this question requires controlling each factor independently, and requires ground truth that is grounded in the rendered pixels rather than only in a scene description.

GRAFIQ, short for *Graphic Intelligence Quotient* (a graphics–literacy IQ for VLMs), addresses this requirement with a deterministic synthetic–rendering pipeline that produces pairwise forced–choice QA items along four axes of image formation: geometry (3D shape, depth, and surface orientation), materials (albedo, gloss, and view–dependent reflectance), lighting (direction and shading), and viewing correspondence across camera changes. For each pair, only the factor under test is varied between the two alternatives; all other factors are matched. The Blender pipeline emits an RGB image together with the render pass that carries the causal signal for the task (depth, glossy–direct, surface–normal, or diffuse–direct), along with a per–object index pass used for masking. A second pipeline stage reads the pass, computes a per–object score under the object mask, and discards any item whose pass–derived answer disagrees with the metadata–derived answer. The resulting *pixel–verified* label ensures that the correct answer is supported by what the VLM actually sees.

Empirically, we study two tracks: *recognition*, in which seven VLMs answer viewer–relative `Left/Right` questions on a 480-item split (Section 4.1), and *inverse generation* (coding), in which selected models complete scaffolded Blender Python on a 120-item subset scored by headless execution yield (Section 4.2).

Figure 1 shows one pixel–verified item per task together with its natural–language question, the correct viewer–relative `Left/Right` answer, and the pass–level evidence used to verify it; the four axes are summarised below.

Benchmark axes. **Geometry** covers 3D shape, depth ordering, and surface orientation. **Materials** covers albedo–versus–gloss reasoning and view–dependent appearance. **Lighting** covers shading and the direction of illumination. **Viewing and correspondence** covers tracking objects and scene identity across camera changes. The released benchmark implements the first three axes; viewing and correspondence is planned as an extension using the same rendering and indexing machinery (Section 5).

Research questions. We structure the empirical analysis around three questions. **RQ1**: Do contemporary VLMs exceed chance on controlled pairwise comparisons in which exactly one graphics factor differs between the two objects? **RQ2**: Which axis is hardest under this protocol? We hypothesise that materials and light–direction judgments are more difficult than depth and roughness magnitude comparisons. **RQ3**: How sensitive are scores to scene context when the same object pair is placed in richer environments while the pass-level signal is held comparable? Section 4.1 answers this on three procedural–backdrop tiers (Studio, Furnished, Outdoor); Section 5 outlines further backdrop scaling toward full–room procedural interiors.

Contributions. We contribute: (i) a deterministic, reproducible two–stage data–generation pipeline in Blender and Python; (ii) four tasks spanning geometry, materials, and lighting, each with pixel–verified ground truth and three natural–language question variants; (iii) a public dataset of procedural primitives in twelve procedural environments, with a documented path to denser meshes and Infinigen-backed rooms (Section 5); (iv) a curated 480-item recognition split; (v) a seven-model recognition study under a shared protocol (Section 4.1); and (vi) an inverse-generation coding benchmark on a 120-item subset that tests whether models can produce runnable Blender code from reference renders (Section 4.2).

2 Related Work

Our benchmark sits at the intersection of three research threads: perceptual probing of large vision–language models, benchmarks that target physical or geometric reasoning, and synthetic rendering pipelines that provide unambiguous ground truth. We survey each in turn and highlight where GRAFIQ departs from prior work.

Perceptual probing of VLMs. General–purpose VLM evaluation suites such as MMBench [14], MMMU [19], and SEED–Bench [13] aggregate dozens of downstream tasks into a single leaderboard number. While useful for tracking overall model progress, most of their items probe semantics or world knowledge rather than image formation: a model can score well on them while still mis–reading depth, shading, or material. Earlier visual–question–answering datasets such as VQA [1], GQA [9], and NLVR2 [18] similarly emphasise recognition and relational reasoning over appearance. A more pointed line of work explicitly probes perception. BLINK [6] shows that otherwise strong VLMs fail on basic perceptual primitives such as depth ordering, multi–view correspondence, and reflectance estimation, suggesting that semantic competence does not imply perceptual competence. "What's 'up' with VLMs?" [11] shows that even today's best systems confuse basic spatial prepositions (above, below, left of) on ordinary natural images, and SpatialVLM [3] proposes targeted training data to close this gap. GRAFIQ is in the same perceptual–probing spirit but narrows the scope further: rather than testing a broad spectrum of perceptual abilities, each of our items isolates exactly one of four image–formation factors (depth, roughness, normal, light direction) and verifies the label against the exact render pass.

Physics, geometry, and material reasoning. A growing number of benchmarks focus on whether VLMs can reason about physics and the 3D world. PhysBench [4] benchmarks VLMs on physical–world properties such as dynamics, static properties, and object relations,

and shows that scaling alone does not close the gap to human performance. CATER [7] used controlled rendered videos to isolate compositional reasoning about object motion under occlusion. On the materials side, intrinsic–image decomposition datasets such as Intrinsic Images in the Wild [2] collect pair–wise reflectance comparisons on real photographs, and retro–reflectance benchmarks have been used to test whether models can separate shading from albedo. Geometric perception has been studied separately in multi–view matching, novel–view synthesis evaluation, and depth estimation datasets. These works collectively show that *specific* graphics factors, in isolation, are hard for current models. GRAFIQ complements them by giving the VLM the full rendered image and asking it to reason about the factor that produced its appearance, in a forced–choice format that can be graded automatically and does not require a model to regress to a physical quantity.

Controlled synthetic rendering benchmarks. Synthetic rendering has a long history as a way to obtain unambiguous ground truth for visual reasoning. CLEVR [10] established that a deterministic Blender pipeline plus template–based QA can serve as a reproducible "unit test" for compositional visual reasoning, and has been reused for many follow–up datasets that extend it along language, visual, or physics axes. Kubric [8] generalises this recipe to a scalable synthetic scene generator with dense annotations (optical flow, depth, segmentation, correspondences) and has been used to build both perception and physics benchmarks. More recent work such as 3DSRBench [15] uses synthetic scenes specifically to evaluate 3D spatial reasoning in VLMs. GRAFIQ borrows two ideas from this tradition: template–based forced–choice questions, which make grading trivial and phrasing–robust, and a deterministic scripted renderer, which makes every item reproducible from a seed and a short descriptor. We depart from these benchmarks in one important respect: whereas prior datasets derive labels from *scene metadata* ("the cube is at depth z"), we derive labels from the *render passes* actually shown to the model, so the correct answer is guaranteed to be supported by the pixels the VLM sees. This pixel–verification step catches cases where the scene metadata is correct but the rendered image is ambiguous (e.g. the "closer" object is occluded), and we believe it is the key design choice that makes GRAFIQ a faithful test of graphics understanding rather than of scene–graph memorisation.

3 Benchmark Construction

This section specifies how benchmark instances are produced and verified. GRAFIQ follows a two–stage pipeline (Figure 2): a headless Blender stage constructs the scene, places two objects, varies exactly one task–dependent graphics factor, and exports RGB together with auxiliary passes; a Python stage reads those passes under per–object masks, checks consistency with scene metadata, instantiates a question template, and writes a pixel–verified record. Items failing the pass–metadata agreement test are discarded, so every retained label is supported by the pixels shown to a model.

3.1 Scene and Asset Control

Each instance places two foreground objects in view and varies exactly one graphics factor (Section 3.2). We prioritise **controlled primitives**: twenty meshes generated procedurally (cube, sphere, Suzanne, tetrahedron, …) without baked textures, so that material and lighting tasks cannot be solved from accidental albedo cues in asset files. The same object pairs are re–rendered under **twelve procedural environments**: four studio–style layouts (*plain*, *studio*, *room*, *gradient*), four furnished rooms (*bedroom*, *kitchen*, *living_room*, *office*), and four outdoor surfaces (*outdoor*, *desert*, *snow*, *concrete*), each authored with Blender shader nodes and no external HDRIs. The same rendering stack optionally accepts downloaded meshes (e.g. Objaverse [5]) under neutralised materials; the recognition study in Section 4.1 uses primitives only. Table 1 lists tasks and evaluation counts. Figure 6 in Section 5 summarises the backdrop realism axis: what the released benchmark covers today versus the full-room procedural interiors we plan next.

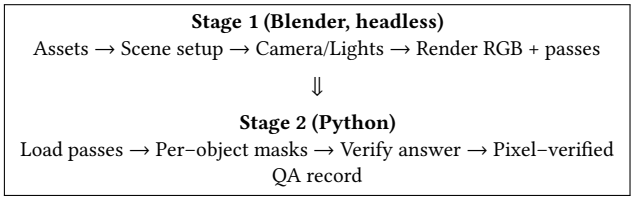


Figure 2: Two–stage data–generation pipeline. Stage 1 produces the rendered image and the auxiliary passes; Stage 2 verifies the ground–truth answer against the pixels actually observed.

3.2 QA Formulation

For each task, Stage 1 varies a single graphics factor between the two objects, while Stage 2 reads the corresponding auxiliary render pass (depth, glossy–direct, surface normal, or diffuse–direct), computes a per–object statistic under Blender's `IndexOB` mask, and accepts the item only if the pass–derived answer matches the scene metadata. Objects in group A use pass indices 1..N and group B use 101..100+M so multi–mesh assets never alias between sides.

Implementation details. Three implementation details were required for the pass–level verification to be reliable. First, Blender normalises the `IndexOB` pass when it is written as PNG, destroying exact index values; we therefore write the index pass as 32–bit float EXR and recover integer indices at read time via $\lfloor \text{value} + 0.5 \rfloor$. Second, for roughness tasks the importer strips all imported GLB materials and substitutes a single Principled BSDF with the target roughness, so that the ground–truth roughness value corresponds exactly to the shader used in the render; for non–roughness tasks all objects receive a neutral grey material to remove albedo as a confound. Third, Blender 4.x writes depth EXRs with a single channel named `V` rather than `R`, so the reader selects the first channel in sorted order to remain compatible across versions.

Question phrasing and design rationale. For each task we maintain three natural–language question variants and select among them deterministically using `seed` mod $|\mathcal{T}|$, where $\mathcal{T}$ is the template set. This prevents a model from overfitting to a single phrasing while keeping each item fully reproducible. Every final QA record is a forced–choice pair over the options `["Left", "Right"]`, stored together with a natural–language `answer` and a letter `answer_key`

Table 1: Tasks in the recognition split (Section 4.1): $4\times120 = 480$ pixel–verified items, one varied graphics factor per task.

Task	Category	Varied factor	n_{eval}
`depth_comparison`	Geometry	Object placement depth	120
`roughness_comparison`	Materials	Surface roughness / gloss	120
`normal_orientation`	Geometry	Object yaw	120
`light_direction`	Lighting	Key light direction	120
Total			**480**

so that either format can be used at evaluation time. We intentionally keep every question short, literal, and unambiguous; for example, a `depth_comparison` prompt reads "*From your viewpoint looking at this image, which object is closer to the camera — the one on your left or the one on your right?*". Reading–comprehension difficulty is therefore explicitly excluded as a confound, which isolates the graphics–perception signal that GRAFIQ is designed to measure; any failure on a task should reflect a breakdown in the model's visual understanding rather than in its interpretation of the question.

Verification filter. Pixel agreement is necessary but not sufficient: a verifier metric can be correct while the cue it measures is invisible to a viewer. We therefore add two task–specific drop rules so that residual difficulty in the released benchmark reflects genuine model failure rather than ill–posed evidence. The first targets `light_direction`: items with a per–object brightness ratio satisfying $|\overline{L_A}/\overline{L_B} - 1| < 0.20$ are dropped, since gaps below $\sim$ 20% sit at or below the just–noticeable–difference floor for uniform–shaded primitives on a textured ground, and an attentive human cannot read them reliably from RGB. The second targets `normal_orientation`: items whose object pair contains a rotationally–symmetric primitive (sphere, UV sphere, cone, ring, cylinder, capsule, or disc) are dropped, because such shapes are invariant under yaw about their principal axis and the rendered image therefore carries no readable yaw signal even though a metadata rotation difference exists. On the released split, Stage 2 verified 701 candidate items; the filter dropped 88 (56 small–gap and 32 symmetric–pair removals) and the remaining 613 were subsampled to a balanced 4×120=480 split. The filter only touches `light_direction` (211→155) and `normal_orientation` (197→165); `depth_comparison` and `roughness_comparison` are unchanged.

3.3 Inverse scene synthesis

In addition to discrimination items, we define an **inverse** formulation: given a reference RGB render, a vision–language model must produce Blender Python that reproduces layout, camera, lighting, and backdrop closely enough to run to completion in a headless renderer. The pilot uses a **scaffolded prompt**: the model sees the reference image together with a partial script that already loads the two foreground GLBs, and completes camera, lighting, and environment code (an image-only variant is possible but was not used for the numbers in Section 4.2). Verification is purely **executional**—whether Blender terminates without error and emits a frame—rather than pixel–aligned equality to the reference pass. This branch targets complementary skills (programmatic control of a complex API under version drift); empirical results appear in Section 4.2, alongside recognition accuracy in Section 4.1.

3.4 Defining "Left" and "Right" from the Viewer's Perspective

Because every question is a forced choice between "Left" and "Right", the benchmark is only well–defined if those two labels have a single, unambiguous meaning that is the *same* for the VLM and for the ground–truth verifier. In GRAFIQ we anchor both sides to *the viewer's viewpoint on the rendered image*, and we derive the label from the object's pixel position rather than from its position in the 3D scene graph. This choice is deliberate and was prompted by a real bug in an earlier version of the pipeline, so we document it here as part of the benchmark design.

Why world coordinates are not enough. Internally, the renderer places the first object ("A") at world x=−0.9 and the second object ("B") at world x=+0.9, with the camera at $(0, 8, 1)$ looking toward the origin and $+Z$ up. Under this camera, the camera's "right" direction in world space is world $-X$, so world $+X$ actually projects to the *left* half of the image and world $-X$ projects to the *right* half. An early version of the QA generator mapped $A \mapsto$ Left and $B \mapsto$ Right from these world coordinates; when we then evaluated Qwen2.5–VL on the resulting benchmark the model scored 31.7% on depth comparison and 28.6% on roughness, i.e. *significantly below* the 50% chance rate, because every "Left"/"Right" label in the gold data was silently flipped relative to what a human viewer would assign to the same image. The benchmark was therefore technically consistent with its own scene graph but inconsistent with its own pixels.

Pixel–centroid labelling. The fix is to never use world coordinates to assign "Left" / "Right". For each item, the verifier reads the `IndexOB` mask of each object and computes the mean image–column of its pixels (its centroid c_x in the rendered frame). The object with the smaller c_x is labelled "Left" and the other "Right", independently of which scene–graph slot (A or B) it occupies and independently of the underlying camera rig. This rule is implemented in `compute_side_map(...)` in `generate_qa_multi.py` and is run before every question template is instantiated, so the stored `answer` field always refers to the side on which the correct object is actually rendered.

Matching the question wording to the label. To keep the textual prompt consistent with this pixel–level definition, all three template variants per task explicitly tie the spatial terms to the viewer. `depth_comparison` asks "*From your viewpoint looking at this image, which object is closer …*"; `roughness_comparison` asks "*From your*

viewpoint, which of the two objects has a lower surface roughness …"; `normal_orientation` asks "*… which object is rotated more toward the camera — the one on your left or the one on your right?*"; `light_direction` asks "*Looking at this image, which object is more brightly illuminated …*". A VLM that understands the image should therefore interpret "Left" and "Right" in exactly the same way the verifier does, and any residual answer–side bias can be cleanly attributed to the model rather than to a mismatch between the question and the label.

4 Evaluation

This section has two parts: **Recognition Evaluation** (Section 4.1) on the full 480-item primitive split in Table 1, and **Inverse Generation Evaluation** (Section 4.2) on a 120-item subset for coding-style inverse benchmarking, aligned with the inverse formulation in Section 3.3. The same rendering stack can ingest non-primitive meshes (e.g. Objaverse [5]) and full-room procedural scenes (Section 5); we fix asset source and backdrop family here so model differences are not confounded by those axes.

4.1 Recognition Evaluation

Stimuli and protocol. Figure 1 (first page) shows one verified example from each of the four implemented tasks in a single 2×2 block: the rendered RGB image, the natural–language question, the correct `Left`/`Right` answer, and the pass–level evidence that makes the label unambiguous. Together the four panels span geometry (depth and normal), materials (roughness), and lighting.

Across all four tasks, the protocol is identical: the model receives a single rendered image and a short viewer–relative question, and must answer with exactly one of `Left` or `Right`. Labels are accepted only when they agree with the pass–derived verifier, so the benchmark remains grounded in the rendered evidence rather than in scene metadata alone. This uniform setup makes the per-task comparison in Table 2 directly interpretable.

Models and inference. To measure whether GRAFIQ separates contemporary VLMs rather than merely serving as a single-model sanity check, we evaluate **seven models** on the full 480-item split: Qwen2.5–VL–7B–Instruct and Qwen3–VL–8B–Thinking locally with vLLM [12], and GLM–4.5V, GPT–4o–mini, GPT–5–mini, Gemini 3.1 Pro (Thinking), and Claude Opus 4.5 through OpenRouter. All runs use greedy decoding, and uncertainty is reported with percentile bootstrap confidence intervals together with exact binomial tests against the 50% chance rate. For reasoning models, the decision is extracted from the final answer segment rather than from intermediate thinking traces, preventing truncation artifacts from being counted as task errors.

Tables 2 and 3 summarize the main comparative results and Left/Right diagnostics. On **RQ1**, all seven models score above chance overall, but the margin varies substantially: Gemini 3.1 Pro leads at 74.8%, followed by GPT–5–mini at 69.6% and GLM–4.5V at 69.2%, while GPT–4o–mini is weakest at 57.3%.

On **RQ2**, the same broad task ordering appears across model families: `roughness_comparison` and `depth_comparison` are reliably the easiest axes, while `normal_orientation` and `light_direction` remain much harder and often statistically indistinguishable from chance. Gemini is the strongest model on depth (94.2%), roughness (93.3%), and normal orientation (63.3%), while Qwen3 is the strongest on light direction (60.0%). This pattern is consistent with the hypothesis that direct pairwise magnitude judgments are easier for current VLMs than inference over shading and viewpoint cues that require a more stable internal 3D frame.

A positional–bias analysis. Table 3 makes the parse and answer–side statistics explicit. The gold Left fraction on the released split is 53.8%, but several models deviate substantially in their predicted Left rate. Qwen3 is the clearest case: it emits Left 73.3% of the time, +19.5 points vs. gold, which can depress raw accuracy on slices where the correct side skews Right even when balanced accuracy is more stable. Conversely, GPT–5–mini answers Left only 40.0% of the time, and Claude Opus 4.5 only 44.4%. These asymmetries help explain uneven performance on `normal_orientation` or `light_direction` relative to depth or roughness. The results therefore reinforce that GRAFIQ is not only measuring whether a model can rank a pair correctly, but also whether it can do so without a brittle pretraining prior over answer side.

Table 4 reports balanced accuracy on the same predictions. The leaderboard ordering is preserved at the top—Gemini and GPT–5–mini remain the two strongest models—but the bias–robust view changes the relative position of two models in informative ways. GPT–5–mini overtakes GLM–4.5V once side–bias is removed (mean 72.3% vs. 69.0%), even though it trails GLM by 0.4 points on raw accuracy; conversely, Qwen3–VL drops from third–from–bottom on raw accuracy to last on balanced accuracy (59.8%), which is consistent with its +19.5pp Left–bias gap in Table 3. These shifts confirm that part of the raw–accuracy spread between models is being driven by L/R prior alignment with the gold distribution rather than by graphics perception per se.

Why are normal and light direction near chance? The two harder axes remain near 50% for almost every model even after the verification filter (Section 3.2) removed the most obviously ill–posed items. We audited this regime directly. First, label correctness is not the issue: independently re–deriving the gold answer from the raw render passes and the pixel–centroid side map agrees with the released label on every checked item, on both the `normal_orientation` and `light_direction` audit subsamples. Second, balanced accuracy on these tasks (Table 4) sits in the 44–63% range across all seven models—models are roughly *guessing*, not systematically inverting; a flipped–label artifact would yield balanced accuracy deeply below 50% (e.g. 10–30%), which we never observe. Third, two structural causes remain after filtering. (i) The verifier integrates a normal– or diffuse–direct pass over each object's mask, but the natural *viewer cue*—the peak specular highlight or the brightest pixel—agrees with the gold only 55.6% of the time on `light_direction` and 44.0% on `normal_orientation` on a held–out audit. A model that "looks at the highlight" is therefore close to chance even though the verifier's mask–mean is correct. (ii) For items whose evidence gap survived the threshold but is only modestly above it, the rendered RGB still carries little visible signal compared to the depth or roughness pairs. The residual sub–chance behaviour is therefore a property of the *cue*—mean over the mask versus what an image–based reader can read off of pixels—rather than of mislabelling, and is the cleanest direction for future work on these two axes.

Table 2: Recognition accuracy on the n=480 split. Bold = above binomial chance at $p<0.05$ vs. 50%. All = overall accuracy with 95% bootstrap CI; D/R/N/L = per–task % for `depth`, `roughness`, `normal`, `light` (n=120 each).

Model	All	D	R	N	L
Qwen2.5–VL–7B-Instruct	**64.0%** [59.6, 68.3]	**62.5%**	**89.2%**	50.0%	54.2%
Qwen3–VL–8B-Thinking	**60.4%** [56.0, 64.8]	48.3%	**77.5%**	55.8%	**60.0%**
GLM–4.5V-Thinking	**69.2%** [65.0, 73.3]	**84.2%**	**80.0%**	54.2%	58.3%
GPT–4o–mini	**57.3%** [52.7, 61.9]	48.3%	**77.5%**	45.8%	57.5%
GPT–5–mini-Thinking	**69.6%** [65.4, 73.5]	**85.8%**	**85.0%**	**60.8%**	46.7%
Gemini 3.1 Pro-Thinking	**74.8%** [70.8, 78.8]	**94.2%**	**93.3%**	**63.3%**	48.3%
Claude Opus 4.5-Thinking	**60.2%** [55.8, 64.6]	**74.2%**	**71.7%**	45.0%	50.0%

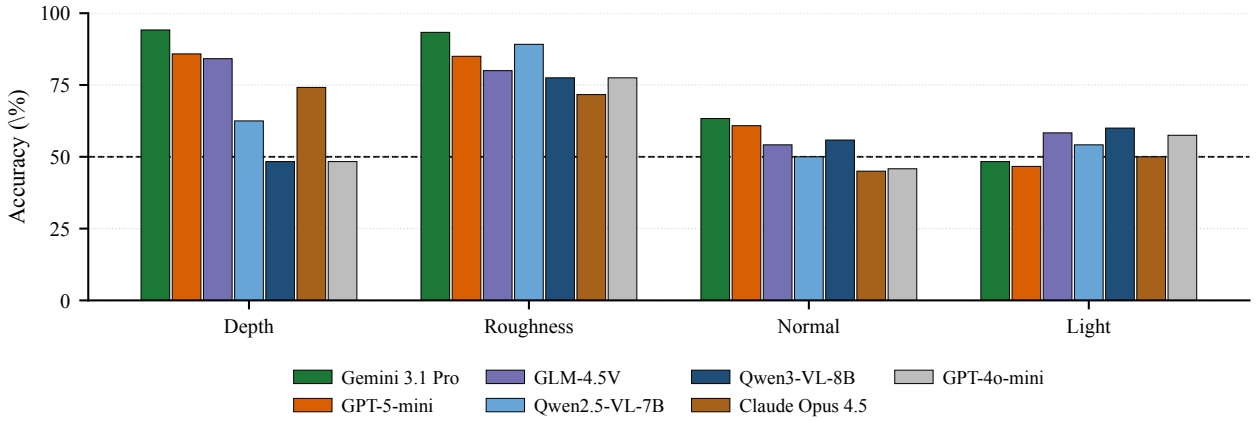


Figure 3: Per–task recognition accuracy on the n=480 split. Within each task group, bars are ordered by overall model accuracy (best on the left). The dashed line marks the 50% binomial chance rate.

Table 3: Left/Right diagnostics on the same n=480 split: valid answer extraction rate and the fraction of predictions equal to `Left`. Gold Left is the benchmark fraction of `Left` labels 258/480 ≈ 53.8%.

Model	Parse	Pred. L	Gold L
Qwen2.5–VL–7B-Instruct	100.0%	64.4%	53.8%
Qwen3–VL–8B-Thinking	100.0%	73.3%	53.8%
GLM–4.5V-Thinking	99.8%	49.0%	53.8%
GPT–4o–mini	100.0%	59.0%	53.8%
GPT–5–mini-Thinking	100.0%	40.0%	53.8%
Gemini 3.1 Pro-Thinking	100.0%	49.8%	53.8%
Claude Opus 4.5-Thinking	100.0%	44.4%	53.8%

Effect of scene backdrop (RQ3). We re–bin the same 480 predictions by the three backdrop tiers used in Section 3.1: *Studio* (`plain`, `studio`, `room`, `gradient`; n=158), *Furnished* (`bedroom`, `kitchen`, `living_room`, `office`; n=152), and *Outdoor* (`outdoor`, `desert`, `snow`, `concrete`; n=170). Table 5 reports per–tier accuracy.

Two findings: *(a)* adding scene context does not hurt graphics recognition on average—no model collapses on the richer Furnished or Outdoor tiers despite the increase in distractor texture and geometry; *(b)* two of the strongest models (Gemini 3.1 Pro and GPT–5–mini) actually score +13.9 and +14.3 points higher on *Furnished* than on *Studio*. We interpret this as a signal that those models exploit scene priors (a textured floor or wall provides reliable ground–plane and lighting–direction context), rather than degrading from added clutter. GPT–4o–mini is the one model that drops noticeably on *Outdoor* (50.0%, vs. 66.4% Furnished), consistent with its known weakness on materials and lighting under unfamiliar shaders. The release benchmark therefore already separates contextual robustness from per–task accuracy, and partially answers RQ3: scene context tends to help, not hurt, modulo per–family reasoning capacity.

Table 4: Balanced accuracy per task on the n=480 split, defined as $\frac{1}{2}\cdot(\text{acc}\,|\,\text{gold=Left} + \text{acc}\,|\,\text{gold=Right})$. Mean averages the four tasks. Computed from the same predictions as Table 2.

Model	D	R	N	L	Mean
Qwen2.5–VL–7B-Instruct	69.2%	88.4%	50.3%	52.7%	65.1%
Qwen3–VL–8B-Thinking	54.1%	77.2%	56.6%	51.1%	59.8%
GLM–4.5V-Thinking	83.2%	84.1%	54.0%	54.7%	69.0%
GPT–4o–mini	57.2%	82.7%	45.6%	55.3%	60.2%
GPT–5–mini-Thinking	85.7%	87.9%	60.8%	54.7%	**72.3%**
Gemini 3.1 Pro-Thinking	94.8%	93.2%	62.9%	46.7%	**74.4%**
Claude Opus 4.5-Thinking	76.1%	76.6%	44.3%	49.3%	61.6%

Table 5: Per–tier accuracy on the n=480 split, all four tasks combined. Cells are acc% (c/n), with n the number of items in that tier. Bold = each model's best tier.

Model	Studio (n=158)	Furnished (n=152)	Outdoor (n=170)
Qwen2.5–VL–7B-Instruct	63.9% (101)	63.8% (97)	**64.1% (109)**
Qwen3–VL–8B-Thinking	**62.0% (98)**	59.9% (91)	59.4% (101)
GLM–4.5V-Thinking	**71.5% (113)**	66.4% (101)	69.4% (118)
GPT–4o–mini	56.3% (89)	**66.4% (101)**	50.0% (85)
GPT–5–mini-Thinking	64.6% (102)	**78.9% (120)**	65.9% (112)
Gemini 3.1 Pro-Thinking	70.3% (111)	**84.2% (128)**	70.6% (120)
Claude Opus 4.5-Thinking	61.4% (97)	**63.8% (97)**	55.9% (95)

Table 6: Inverse–generation coding benchmark: headless Blender execution yield on n=120 samples per model. An item counts as executable if Blender exits with status zero and writes a reconstruction frame.

Model	Executable	Rate
Gemini 3.1 Pro	120 / 120	100.0%
Claude Opus 4.5	97 / 120	80.8%
GPT–4o–mini	47 / 120	39.2%
GPT–5–mini	40 / 120	33.3%
Qwen3–VL–8B	17 / 120	14.2%

Interpretation. This comparison changes the project status in an important way: the benchmark is no longer just a single-model sanity check. It now separates a seven-model leaderboard with clear differences by family, reasoning mode, and bias profile, while keeping the underlying task format fixed. Reasoning has a measurable effect: averaging across the four tasks, the five thinking models reach 66.8% overall versus 60.6% for the two non-thinking models—a +6.2pp gap on the same protocol. That makes the next stage of work mostly about increasing coverage and sample size rather than about inventing a new evaluation protocol.

4.2 Inverse Generation Evaluation

Recognition results (Table 2) show that `depth_comparison` is the most reliably above–chance axis for most models, i.e. *reading* depth relations from our renders is comparatively well behaved. We therefore pilot the inverse formulation of Section 3.3 on a **120-item subset** of showcase renders—ten foreground pairs crossed with twelve procedural backgrounds—before investing in the same procedure for roughness, normals, or light direction, where recognition accuracies are lower and noisier. Each item pairs a fixed reference frame from the public split with a model-generated `script.py` under the scaffolded prompt. Verification uses headless Blender 4.4.3: an item counts as **executable** if the process exits with status zero and writes a reconstruction frame. Reported rates are *execution yields*, not pixel match to the reference.

Table 6 and Figure 4 summarise the full 120–sample batches we have completed to date. Gemini 3.1 Pro (Thinking) attained the highest execution yield (100% after a minor systematic typo fix—it consistently prefixed the standard `mathutils` module with `bpy.`, so its yield before the fix was only ∼ 25%); the underlying scripts were spatially and logically correct in every sample we audited.

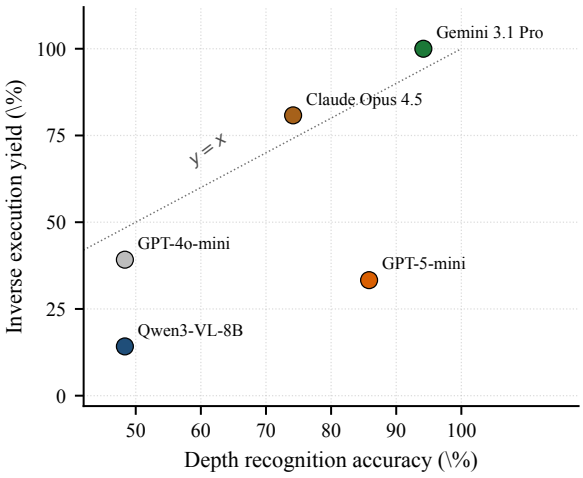


Figure 4: Recognition accuracy on `depth_comparison` versus inverse–generation execution yield. Each marker is one model; the dotted line is y=x. Marker colours match Figure 3.

Claude Opus 4.5 also performs well at 80.8%. Other models lag despite strong *recognition* scores on depth: GPT–5–mini reaches 85.8% on `depth_comparison` but only 33.3% executable on the inverse subset, a concrete instance where a strong graphics *recogniser* is not a strong *inverse* generator under Blender 4.x. Qwen3–VL–8B similarly struggles (14.2%) due to camera–setup errors and asset hallucinations.

Failure modes. Manual inspection of `stderr` in failed rows isolates four recurring patterns that span model families. (i) **Principled BSDF socket renames**: models index the `Specular` input of the BSDF, which raises `KeyError` because the socket was renamed to `Specular IOR Level` in Blender 4.0. (ii) **Invalid light–object construction**: models call `objects.new` with `type='LIGHT'`, which is rejected because `type` is not a valid kwarg; the API instead requires the caller to first build a light datablock via `lights.new` and pass it as `object_data`. (iii) **Response truncation under verbose reasoning**: GPT–5–mini routinely hits the 4096–token output limit mid–statement, producing a syntactically incomplete file that fails to parse. (iv) **Deprecated attributes** such as `cycles_visibility` that were moved or removed across Blender major versions. Failure modes (i) and (ii) are deterministic API drift rather than perceptual failure: a model that misread the scene would still produce a runnable but visually inaccurate render, whereas these scripts crash before producing any frame.

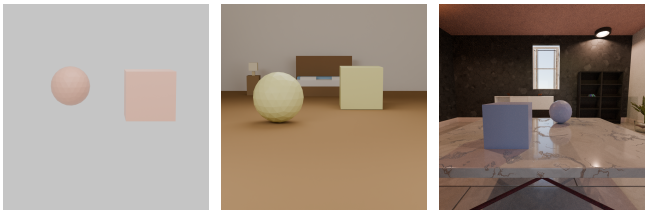
Figure 5 illustrates variation in geometry and backdrop among successful pairs; many other subset rows still fail for the API issues noted above, with full `stderr` traces retained per item in the evaluation logs.

5 Limitations and Next Steps

The current report establishes a working benchmark and a meaningful seven–model comparison, but several limitations remain.

Figure 5: Six reference scenes from the inverse coding subset and the corresponding model reconstructions under headless Blender 4.4.3. The first column is the reference render; the remaining columns show executable outputs from each model, which are not pixel–matched to the reference. Overall execution yields are in Table 6.

Scene backdrop roadmap. The released recognition and inverse pilots are rendered against **procedural environments** that already add furniture, materials, and outdoor shaders, but they stop short of full architectural interiors generated as first-class content. Figure 6 situates the gap: panel (a) shows the simplest neutral world; panel (b) typifies what we ship today (the same cube–and–sphere pair in a furnished bedroom); panel (c) is an *illustrative*



(a) Neutral studio world. **(b) Furnished procedural room (released axis).** **(c) Target: full-room procedural interior (future work).**

Figure 6: Backdrop realism, from left to right: a neutral studio world, a furnished procedural room from the current split, and a representative full–room procedural interior (Infinigen).

Infinigen [16, 17] interior—the realism axis we want to add so that models are tested under richer room structure than (b) alone.

Dataset scale and diversity. The current leaderboard uses the 480-item primitive split only. The pipeline already produces hundreds of items in minutes, and the next target is a few thousand items per task family, optionally including downloaded meshes and Infinigen-backed interiors, so the recognition protocol of Section 4.1 can be rerun at scale without changing the evaluation harness.

Procedural indoor rooms (Infinigen). Beyond Figure 6(c), we plan to integrate Infinigen [16, 17] (vendored under `tools/infinigen/`): generate an Infinigen–Indoors `scene.blend` with a named anchor (e.g. `center_table`), open it in Blender, place the two candidate objects on that surface with the same pass–based verification, and re–render. This axis is **future work**: it requires a reliable headless Infinigen/Blender 4.2 toolchain, enough RAM for coarse scene generation, and batched `scene.blend` production before we can ship a balanced split alongside the primitive corpus.

A fifth category: Viewing & Correspondence. The proposal lists *Viewing & Correspondence* as one of the four axes, but we have not yet implemented it. We plan to reuse the existing scene constructor with two different cameras and a matching task that asks either (i) "do these two images show the same scene?" or (ii) "which object in image B corresponds to object X in image A?". The same `IndexOB` pass gives us the per–object correspondence for free.

VLM evaluation. The evaluation interface is deliberately minimal: each model receives an image, a question, and the two response options `Left` and `Right`. That part is now substantially complete: the current report covers two local open models and five API models under a common harness, with valid parse rates and saved predictions. The remaining evaluation work is to (i) add stronger open baselines such as Qwen2.5–VL–72B and InternVL variants, (ii) rerun the leaderboard on the larger cross–tier benchmark once the dataset is scaled up, and (iii) if time allows, collect a small human baseline as a reference ceiling.

Risks. Two open risks remain. First, some tasks may become trivially solvable for the strongest models—Gemini reaches 94.2% on `depth_comparison` and 93.3% on `roughness_comparison` on the present split, leaving little headroom—in which case we will harden them by narrowing the gap between the two objects' ground–truth values, following BLINK's recipe of filtering items with very large "signal gaps". Second, the residual sub–chance behaviour on `normal_orientation` and `light_direction` (Section 4.1) suggests that the verifier metric—a mean over the per–object mask—does not always coincide with the visual cue a viewer can read from a single RGB frame. Future revisions of these two tasks should either expose the verifier cue more saliently in the render (e.g. a sharper key–light gradient for `light_direction`, asymmetric primitives only for `normal_orientation`) or switch the verifier to a peak–based statistic (max or 95th–percentile pass intensity) that better matches what a model is likely to attend to.

6 Summary

We present a reproducible, multi–GPU pipeline that turns a controlled set of 3D assets into pixel–verified forced–choice items across four graphics axes, together with a seven-model recognition study on a 480-item split and a 120-item inverse-generation coding benchmark subset (Section 4.2). Rendering, QA generation, pass-level verification, and evaluation logging are automated end to end. Section 5 discusses scaling the dataset, richer indoor scenes, and additional baselines as natural extensions of the same harness.

Acknowledgments

We thank the CS 184/284A course staff for feedback on the project proposal. This project builds on the open Blender ecosystem, the Infinigen [16, 17] procedural–generation engine, and the ACM `acmart` template. We additionally acknowledge the use of Anthropic's Claude and the Cursor IDE as coding assistants during the implementation of the pipeline and the preparation of this report.

References

[1] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C. Lawrence Zitnick, and Devi Parikh. 2015. VQA: Visual Question Answering. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*.

[2] Sean Bell, Kavita Bala, and Noah Snavely. 2014. Intrinsic Images in the Wild. *ACM Transactions on Graphics (SIGGRAPH)* 33, 4 (2014).

[3] Boyuan Chen, Zhuo Xu, Sean Kirmani, Brian Ichter, Danny Driess, Pete Florence, Dorsa Sadigh, Leonidas Guibas, and Fei Xia. 2024. SpatialVLM: Endowing Vision-Language Models with Spatial Reasoning Capabilities. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[4] Wei Chow, Jiageng Mao, Boyi Li, Daniel Seita, Vitor Guizilini, and Yue Wang. 2025. PhysBench: Benchmarking and Enhancing Vision-Language Models for Physical World Understanding. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[5] Matt Deitke, Dustin Schwenk, Jordi Salvador, Luca Weihs, Oscar Michel, Eli VanderBilt, Ludwig Schmidt, Kiana Ehsani, Aniruddha Kembhavi, and Ali Farhadi. 2023. Objaverse: A Universe of Annotated 3D Objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[6] Xingyu Fu, Yushi Hu, Bangzheng Li, Yu Feng, Haoyu Wang, Xudong Lin, Dan Roth, Noah A. Smith, Wei-Chiu Ma, and Ranjay Krishna. 2024. BLINK: Multimodal Large Language Models Can See but Not Perceive. In *Proceedings of the European Conference on Computer Vision (ECCV)*.

[7] Rohit Girdhar and Deva Ramanan. 2020. CATER: A Diagnostic Dataset for Compositional Actions and Temporal Reasoning. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[8] Klaus Greff, Francois Belletti, Lucas Beyer, Carl Doersch, Yilun Du, Daniel Duckworth, David J. Fleet, Dan Gnanapragasam, Florian Golemo, Charles Herrmann, et al. 2022. Kubric: A Scalable Dataset Generator. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[9] Drew A. Hudson and Christopher D. Manning. 2019. GQA: A New Dataset for Real-World Visual Reasoning and Compositional Question Answering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[10] Justin Johnson, Bharath Hariharan, Laurens van der Maaten, Li Fei-Fei, C. Lawrence Zitnick, and Ross Girshick. 2017. CLEVR: A Diagnostic Dataset for Compositional Language and Elementary Visual Reasoning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).*

[11] Amita Kamath, Jack Hessel, and Kai-Wei Chang. 2023. What's "up" with Vision-Language Models? Investigating Their Struggle with Spatial Reasoning. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP).*

[12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP).*

[13] Bohao Li, Rui Wang, Guangzhi Wang, Yuying Ge, Yixiao Ge, and Ying Shan. 2024. SEED-Bench: Benchmarking Multimodal LLMs with Generative Comprehension. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).*

[14] Yuan Liu, Haodong Duan, Yuanhan Zhang, Bo Li, Songyang Zhang, Wangbo Zhao, Yike Yuan, Jiaqi Wang, Conghui He, Ziwei Liu, Kai Chen, and Dahua Lin. 2024. MMBench: Is Your Multi-modal Model an All-around Player?. In *Proceedings of the European Conference on Computer Vision (ECCV).*

[15] Wufei Ma, Haoyu Chen, Guofeng Zhang, Celso M. de Melo, Alan Yuille, and Jieneng Chen. 2024. 3DSRBench: A Comprehensive 3D Spatial Reasoning Benchmark. In *arXiv preprint arXiv:2412.07825.*

[16] Alexander Raistrick, Lahav Lipson, Zeyu Ma, Lingjie Mei, Mingzhe Wang, Yiming Zuo, Karhan Kayan, Hongyu Wen, Beining Han, Yihan Wang, Alejandro Newell, Hei Law, Ankit Goyal, Kaiyu Yang, and Jia Deng. 2023. Infinite Photorealistic Worlds Using Procedural Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).*

[17] Alexander Raistrick, Lingjie Mei, Karhan Kayan, David Yan, Yiming Zuo, Beining Han, Hongyu Wen, Meenal Parakh, Stamatis Alexandropoulos, Lahav Lipson, Zeyu Ma, and Jia Deng. 2024. Infinigen Indoors: Photorealistic Indoor Scenes using Procedural Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).*

[18] Alane Suhr, Stephanie Zhou, Ally Zhang, Iris Zhang, Huajun Bai, and Yoav Artzi. 2019. A Corpus for Reasoning About Natural Language Grounded in Photographs. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL).*

[19] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, et al. 2024. MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).*